

RAPORT Z PROJEKTU

Gra Breakthrough — Implementacja algorytmu Minimax z cięciami alfa-beta

Sztuczna Inteligencja i Inżynieria Wiedzy — Lista 2

03.05.2026

Technologia: Python 3 | Algorytm: Minimax + cięcia alfa-beta

Heurystyki: 5 strategii (materiał, wyścig, presja, zagrożenie, adaptacyjna)

Pliki: *agents.py* | *state.py* | *heuristics.py* | *game.py* | *gui2.py* | *optimize_weights.py*

Spis treści

1. Cel projektu i opis teoretyczny metody
2. Formalne sformułowanie problemu
3. Architektura systemu — opis modułów
4. Algorytm Minimax
5. Cięcia alfa-beta
6. Heurystyki oceny stanu gry
7. Kluczowe optymalizacje implementacyjne
8. Wersja rozszerzona (dodatkowe funkcjonalności)
9. Wyniki eksperymentalne i anomalie
10. Napotkane problemy implementacyjne
11. Literatura i biblioteki

1. Cel projektu i opis teoretyczny metody

Celem projektu było zaimplementowanie agenta grającego w dwuosobową grę planszową Breakthrough, zdolnego do samodzielnego podejmowania decyzji ruchowych na podstawie przewidywania przyszłych stanów gry. Jako główną metodę przeszukiwania przestrzeni stanów zastosowano algorytm Minimax uzupełniony o optymalizację w postaci cięć alfa-beta.

1.1 Algorytm Minimax

Minimax to klasyczna strategia przeszukiwania drzewa gry stosowana w grach dwuosobowych o sumie zerowej (takich jak szachy, warcaby czy Breakthrough). Kluczowe założenie: obaj gracze grają optymalnie — jeden stara się maksymalizować swój wynik (MAX), drugi go minimalizować (MIN).

Formalna definicja: dla węzła v w drzewie gry wartość $\text{minimax}(v)$ definiujemy rekurencyjnie:

- $\text{minimax}(v) = \text{użyj heurystyki } h(v)$ — jeśli v jest liściem lub osiągnięto limit głębokości d
- $\text{minimax}(v) = \max\{ \text{minimax}(c) : c \in \text{dzieci}(v) \}$ — jeśli v jest węzłem MAX
- $\text{minimax}(v) = \min\{ \text{minimax}(c) : c \in \text{dzieci}(v) \}$ — jeśli v jest węzłem MIN

Ponieważ drzewo gry rośnie wykładniczo (w Breakthrough jeden ruch generuje ok. 16–20 stanów potomnych), niezbędne jest ograniczenie głębokości przeszukiwania parametrem d oraz zastosowanie funkcji heurystycznej oceniającej stany nieterminalne.

1.2 Cięcia alfa-beta

Cięcia alfa-beta stanowią najważniejszą optymalizację algorytmu Minimax. Nie zmieniają wyniku, lecz drastycznie zmniejszają liczbę odwiedzanych węzłów poprzez pominięcie gałęzi, które z pewnością nie wpłyną na ostateczną decyzję.

Parametry: α (alfa) — najlepsza dotychczas znaleziona wartość dla gracza MAX; β (beta) — najlepsza dla gracza MIN.

- Cięcie alfa: gdy wartość węzła MIN $\leq \alpha \rightarrow$ pomijamy pozostałe dzieci (gracz MIN nigdy nie dopuści do tej sytuacji)
- Cięcie beta: gdy wartość węzła MAX $\geq \beta \rightarrow$ pomijamy pozostałe dzieci (gracz MAX nigdy nie dopuści do tej sytuacji)

Efektywność cięć alfa-beta silnie zależy od kolejności przeglądania ruchów. W idealnym przypadku (perfekcyjne sortowanie) złożoność spada z $O(b^d)$ do $O(b^{(d/2)})$, co pozwala przeszukać drzewo dwa razy głębiej przy tej samej liczbie węzłów.

2. Formalne sformułowanie problemu

2.1 Plansza i reprezentacja stanu

Rozgrywka toczy się na planszy 8×8. Każda komórka jest jednym z czterech symboli:

Symbol	Znaczenie
B	Pionek gracza pierwszego (idzie w dół planszy, kierunek: +1)
W	Pionek gracza drugiego (idzie w górę planszy, kierunek: -1)
—	Pole puste
o	Pole, z którego wykonano ostatni ruch (marker pomocniczy)

2.2 Zasady ruchu

- Pionek może poruszyć się o jedno pole prosto do przodu (jeśli to pole jest puste)
- Pionek może poruszyć się po skosie do przodu-lewo lub przodu-prawo (jeśli pole jest puste)
- Bicie pionka przeciwnika jest możliwe WYŁĄCZNIE po skosie — nie ma bicia w linii prostej
- Wygrywa gracz, którego pionek jako pierwszy dotrze do przeciwległej krawędzi planszy

2.3 Drzewo decyzyjne

Wierzchołek: unikalny stan planszy (obiekt BreakthroughState)

Krawędź: legalny ruch gracza generujący nowy stan

Liść: stan terminalny (wygrany) lub węzeł osiągający limit głębokości d

Głębokość d: konfigurowalny parametr, domyślnie 4–5 (kompromis między jakością a czasem)

3. Architektura systemu — opis modułów

Projekt podzielono na sześć modułów realizujących zasadę pojedynczej odpowiedzialności (SRP):

Moduł	Opis
state.py	Klasa BreakthroughState — reprezentacja planszy, generowanie ruchów, sprawdzanie warunków końcowych. Zawiera wszystkie kluczowe optymalizacje wydajnościowe.
agents.py	Klasa MinimaxAgent — implementacja algorytmu Minimax z cięciami alfa-beta. Przyjmuje konfigurowalną funkcję heurystyczną i parametr głębokości.
heuristics.py	Zbiór pięciu funkcji oceniających stan gry: eval_material, eval_race, eval_pressure, eval_threat, eval_hybrid oraz eval_adaptive.
game.py	Klasa GameRunner — kontroler przebiegu całej gry. Obsługuje wejście/wyjście zgodnie ze specyfikacją (STDOUT/STDERR), wczytywanie planszy ze stdin.
gui2.py	Graficzny interfejs użytkownika (tkinter) — wizualizacja prekalkulowanej gry krok po kroku z wyświetlaniem statystyk i ocen heurystycznych.
optimize_weights.py	Symulator turniejowy — rozgrywa mecze każdy z każdym między agentami z losowymi wagami, umożliwiając ewolucyjne szukanie optymalnych parametrów eval_hybrid.

4. Algorytm Minimax — implementacja

Poniżej przedstawiono kluczowe fragmenty implementacji algorytmu z pliku agents.py:

agents.py — Inicjalizacja i punkt wejścia

```
class MinimaxAgent:
    def __init__(self, color, max_depth, heuristic_func,
**heuristic_kwargs):
        self.color = color
        self.max_depth = max_depth
        self.heuristic_func = heuristic_func
        self.heuristic_kwargs = heuristic_kwargs
        self.visited_nodes = 0

    def get_best_move(self, state):
        self.visited_nodes = 0
        # Zaczynamy od węzła MAX (nasz ruch)
        _, best_state = self._minimax(state, self.max_depth,
                                     -math.inf, math.inf, True,
self.color)
        return best_state, self.visited_nodes
```

agents.py — Rdzeń algorytmu Minimax z cięciami alfa-beta

```
def _minimax(self, state, depth, alpha, beta, is_maximizing,
player_color):
    self.visited_nodes += 1

    if depth == 0 or state.is_terminal():
        score = self.heuristic_func(state, player_color,
**self.heuristic_kwargs)
        # Premia za szybką wygraną: im wyższe depth (więcej kroków
zostało),
        # tym wyżej oceniamy wygraną – algorytm natychmiast ją wybiera
        if score > 50000: score += depth * 1000
        elif score < -50000: score -= depth * 1000
        return score, state

    if is_maximizing:
        max_eval = -math.inf
        for child in state.get_possible_moves(player_color):
            eval_val, _ = self._minimax(child, depth-1, alpha, beta,
False, player_color)
            if eval_val > max_eval:
                max_eval = eval_val; best_state = child
                alpha = max(alpha, eval_val)
                if beta <= alpha: break # ← Cięcie alfa-beta
    else:
        min_eval = math.inf
        enemy_color = 'W' if player_color == 'B' else 'B'
        for child in state.get_possible_moves(enemy_color):
            eval_val, _ = self._minimax(child, depth-1, alpha, beta,
True, player_color)
            if eval_val < min_eval:
                min_eval = eval_val; best_state = child
                beta = min(beta, eval_val)
                if beta <= alpha: break # ← Cięcie alfa-beta
```

4.1 Rozwiązanie efektu horyzontu (Horizon Effect)

Klasyczny problem: algorytm widzi wygraną na głębokości d i ocenia ją na 99999. Ale identyczną ocenę dostaje też ruch, który np. najpierw bije pionka wroga (tracąc przy tym rundę), a potem wchodzi na metę. Oba ruchy wyglądają jednakowo.

Rozwiązanie: do oceny wygranej dodawana jest dynamiczna premia $\text{depth} * 1000$. Im wyższe depth (im mniej kroków zostało do wygranej), tym wyżej wyceniany jest ten stan — algorytm zawsze wybiera najkrótszą drogę do zwycięstwa.

5. Cięcia alfa-beta — wyniki eksperymentalne

Implementacja cięć alfa-beta łączona jest z dodatkowym mechanizmem sortowania ruchów w module state.py (system wiaderek). Ruchy wygrywające z biciem są sprawdzane jako pierwsze, co maksymalizuje skuteczność odcięć.

5.1 Porównanie liczby węzłów i czasu (głębokość d=5)

Minimax bez cięć	918 000 węzłów	9.46 s
Minimax z cięciami alfa-beta	65 000 węzłów	0.84 s
Redukcja	~14× mniej węzłów	~11× szybciej

Porównanie Minimax vs Minimax+alfa-beta

Wyjście *STDERR* z terminala: liczba węzłów i czas działania dla obu wariantów
(918 000 → 65 000 węzłów, 9.46s → 0.84s)

```

• (base) PS C:\Users\Maciek\Documents\AI_pwr\zad2-breakthrough> python game.py
-----
      B
W _ _ _ B B B
      B B _ _
      B _ _ B B
      _ _ _ _
      _ _ _ _
      _ o _ _
      _ B _ W W W W
-----
Liczba rund: 73
Zwyciezca: B
Odwiedzone wezly: 918896
Czas dzialania: 9.4579s
Agent B - Glebokosc: 3, Strategia: eval_race
Agent W - Glebokosc: 3, Strategia: eval_race
• (base) PS C:\Users\Maciek\Documents\AI_pwr\zad2-breakthrough> python game.py
-----
      B
W _ _ _ B B B
      B B _ _
      B _ _ B B
      _ _ _ _
      _ _ _ _
      _ o _ _
      _ B _ W W W W
-----
Liczba rund: 73
Zwyciezca: B
Odwiedzone wezly: 65502
Czas dzialania: 0.8426s
Agent B - Glebokosc: 3, Strategia: eval_race
Agent W - Glebokosc: 3, Strategia: eval_race

```


6. Heurystyki oceny stanu gry

Zaimplementowano pięć funkcji oceniających zdefiniowanych w pliku `heuristics.py`. Każda zwraca wynik z perspektywy danego gracza: wynik dodatni = przewaga gracza, wynik ujemny = przewaga przeciwnika, ± 99999 = stan wygrany/przegrany.

6.1 eval_material — Przewaga materialna

Najprostsza i najszybsza heurystyka. Liczy różnicę w liczbie pionków na planszy między graczami. Promuje strategie defensywne i agresywne bicia.

heuristics.py — eval_material

```
def eval_material(state, player):
    b_count = sum(row.count('B') for row in state.board)
    w_count = sum(row.count('W') for row in state.board)
    if player == 'B': return b_count - w_count
    else: return w_count - b_count
```

6.2 eval_race — Wyścig do mety

Szuka najbardziej wysuniętego pionka każdego gracza i porównuje odległości do celu. Wynik = dystans_wroga - nasz_dystans. Promuje szybkie parcie do przodu, ignorując straty materialne.

heuristics.py — eval_race

```
def eval_race(state, player):
    b_farthest = 0
    w_farthest = state.rows - 1
    for r in range(state.rows):
        for c in range(state.cols):
            if state.board[r][c] == 'B': b_farthest = max(b_farthest, r)
            elif state.board[r][c] == 'W': w_farthest = min(w_farthest, r)
    if b_farthest == state.rows - 1: return 99999 if player == 'B' else -99999
    if w_farthest == 0: return 99999 if player == 'W' else -99999
    b_dist = (state.rows - 1) - b_farthest # dystans B do mety
    w_dist = w_farthest - 0 # dystans W do mety
    if player == 'B': return w_dist - b_dist
    else: return b_dist - w_dist
```

6.3 eval_pressure — Presja terytorialna

Przyznaje pionkom premię kwadratową za zajmowanie zaawansowanych pozycji na planszy. Gracz B dostaje $(wiersz+1)^2$, gracz W dostaje $(wiersz_od_dołu+1)^2$. Promuje szerokie parcie całej armii do przodu, nie tylko jednego pionka.

heuristics.py — eval_pressure

```
def eval_pressure(state, player):
    b_score, w_score = 0, 0
    for r in range(state.rows):
        for c in range(state.cols):
            if state.board[r][c] == 'B':
```

```

        b_score += (r + 1) ** 2          # premia kwadratowa za
wiersz
        elif state.board[r][c] == 'W':
            w_score += (state.rows - r) ** 2
        if player == 'B': return b_score - w_score
        else:             return w_score - b_score

```

6.4 eval_threat — Ocena zagrożeń (defensywna)

Analizuje dla każdego pionka: czy jest atakowany przez przeciwnika? Czy jest chroniony przez własnego sojusznika z tyłu? Pionki zaatakowane bez obrony ("wiszące") obniżają ocenę. Promuje grę defensywną i budowanie wzajemnych osłon.

heuristics.py — eval_threat (fragment)

```

# Fragment sprawdzający zagrożenie dla pionka B:
threatened = False
if r + 1 < state.rows: # W atakuje z dołu (idzie w górę)
    if c - 1 >= 0 and state.board[r+1][c-1] == 'W': threatened = True
    if c + 1 < cols and state.board[r+1][c+1] == 'W': threatened = True
protected = False
if r - 1 >= 0: # sojusznik B osłania z tyłu
    if c - 1 >= 0 and state.board[r-1][c-1] == 'B': protected = True
    if c + 1 < cols and state.board[r-1][c+1] == 'B': protected = True
if threatened and not protected:
    b_hanging += 1 # pionek niezabezpieczony

```

6.5 eval_hybrid — Heurystyka łączona

Liniowa kombinacja czterech powyższych heurystyk z konfigurowalnymi wagami α , β , γ , δ . Wagi zostały wstępnie dobrane eksperymentalnie przy użyciu symulatora turniejowego (optimize_weights.py), a następnie ręcznie dostrojone.

heuristics.py — eval_hybrid z optymalizacją warunkową

```

def eval_hybrid(state, player, alpha=1.0, beta=10.0, gamma=2.0, delta=5.0):
    if 'B' in state.board[state.rows-1]: return 99999 if player=='B' else -
    99999
    if 'W' in state.board[0]: return 99999 if player=='W' else -
    99999

    h1 = eval_material(state, player) if alpha != 0 else 0 # optymalizacja:
    skip jeśli waga=0
    h2 = eval_race(state, player) if beta != 0 else 0
    h3 = eval_pressure(state, player) if gamma != 0 else 0
    h4 = eval_threat(state, player) if delta != 0 else 0

    return (alpha * h1) + (beta * h2) + (gamma * h3) + (delta * h4)

```

Optymalizacja: Warunek if alpha != 0 before każdą heurystyką pozwala całkowicie pominąć kosztowną iterację po planszy, gdy dana składowa ma wagę zerową. W przypadku testowania konfiguracji z wyłączonymi składowymi redukuje to czas obliczeń o kilkanaście procent.

6.6 eval_adaptive — Heurystyka adaptacyjna

Innowacyjna heurystyka, która automatycznie dostosowuje wagi w zależności od bieżącego etapu gry. Kluczowy parametr `game_progress` $\in [0,1]$ mierzy, jak blisko mety jest najbardziej wysunięty pionek na planszy.

heuristics.py — eval_adaptive

```
def eval_adaptive(state, player):
    # Oblicz postęp gry na podstawie najbardziej wysuniętego pionka
    b_progress = b_farthest / (state.rows - 1)
    w_progress = ((state.rows-1) - w_farthest) / (state.rows - 1)
    game_progress = max(b_progress, w_progress) # 0.0 = start, 1.0 = finisz

    # Adaptacyjne wagi — płynna zmiana strategii:
    alpha = 15.0 * (1.0 - game_progress) # Materiał: 15 → 0
    (maleje)
    beta = 5.0 + (150.0 * (game_progress ** 3)) # Wyścig: 5 → 155
    (rośnie wykładniczo!)
    gamma = 5.0 * (1.0 - game_progress) # Presja: 5 → 0
    delta = 6.0 * (1.0 - (game_progress * 0.5)) # Zagrożenie: 6 → 3
```

Etap gry	Wagi (α , β , γ , δ)	Strategia
Wczesna gra (progress ≈ 0)	$\alpha=15$, $\beta=5$, $\gamma=5$, $\delta=6$	Obrona, kontrola materiału, budowanie struktury
Środkowa gra (progress ≈ 0.5)	$\alpha \approx 7.5$, $\beta \approx 23.7$, $\gamma \approx 2.5$, $\delta \approx 4.5$	Balans między wyścigiem a obroną
Końcówka (progress ≈ 0.8)	$\alpha \approx 3$, $\beta \approx 101$, $\gamma \approx 1$, $\delta \approx 3.6$	Dominuje wyścig — agent ignoruje straty
Stan krytyczny (progress $\rightarrow 1.0$)	$\alpha \rightarrow 0$, $\beta \rightarrow 155$, $\gamma \rightarrow 0$, $\delta \rightarrow 3$	Pełna panika — tylko pęd do mety

7. Kluczowe optymalizacje implementacyjne

Cztery optymalizacje niskopoziomowe znacząco przyspieszyły działanie algorytmu w porównaniu z naiwną implementacją referencyjną. Poniżej przedstawiono każdą z nich z fragmentem kodu źródłowego.

7.1 Usuwanie znacznika 'o' w czasie $O(1)$

Naiwne podejście wymagało przeszukania całej planszy 8×8 (64 operacje!) przy każdym generowanym ruchu, aby znaleźć i usunąć poprzedni znacznik 'o'. W głęboko zagnieżdżonym drzewie gry to setki tysięcy zbędnych operacji.

Rozwiązanie: obiekt BreakthroughState przechowuje atrybut `last_origin = (r_from, c_from)`, wskazujący dokładną pozycję ostatniego znacznika 'o'. Usunięcie zajmuje jeden dostęp do tablicy — $O(1)$ zamiast $O(n^2)$.

state.py — Optymalizacja 1: $O(1)$ dla usuwania znacznika 'o'

```
# state.py — _create_new_state()

# Stary kod (naiwny) —  $O(N^2)$ :
# for r in range(self.rows):
#     for c in range(self.cols):
#         if new_board[r][c] == 'o': new_board[r][c] = '_'

# ZOPTYMALIZOWANY kod —  $O(1)$ :
if self.last_origin is not None:
    old_r, old_c = self.last_origin
    if new_board[old_r][old_c] == 'o': # bezpieczna weryfikacja
        new_board[old_r][old_c] = '_' # ← jeden dostęp, koniec
else:
    # fallback: tylko przy pierwszym ruchu gry (last_origin=None)
    for r in range(self.rows):
        for c in range(self.cols):
            if new_board[r][c] == 'o': new_board[r][c] = '_'
```

7.2 Sortowanie ruchów metodą wiaderk (Bucket Sort)

Klasyczne podejście: wygeneruj wszystkie ruchy, dodaj priorytety jako krotki, posortuj. Wywołanie `sort()` na tysiącach krotek w każdym węźle drzewa tworzy ogromny narzut obliczeniowy.

Rozwiązanie: zamiast jednej listy + `sort()`, od razu segregujemy ruchy do czterech list (wiaderk) według priorytetu. Na koniec łączymy je w ustalonej kolejności. Zero sortowania, zero krotek — tylko proste `append()` i konkatencja.

state.py — Optymalizacja 2: Bucket Sort zamiast sort()

```
# state.py — get_possible_moves()
```

```
# Cztery 'wiaderka' zamiast jednej listy + sort():
wins_capture = [] # Priorytet 1: ruch wygrywający + bicie
wins_normal = [] # Priorytet 2: ruch wygrywający (prosty)
captures = [] # Priorytet 3: bicie pionka wroga
normals = [] # Priorytet 4: zwykły ruch

# ...[generowanie ruchów - każdy trafia do właściwego wiaderka]...

# Scalanie - O(n), bez sort(), bez krotek:
return wins_capture + wins_normal + captures + normals
```

7.3 Zmienne lokalne zamiast dostępu przez self.

W Pythonie każde odwołanie do atrybutu obiektu przez `self` wymaga dodatkowego wyszukania w słowniku. W zagnieżdżonej podwójnej pętli iterującej po 64 polach planszy, wielokrotnie wywoływanej dla każdego węzła drzewa, ten koszt się kumuluje.

state.py — Optymalizacja 3: zmienne lokalne

```
# state.py - get_possible_moves()

# ZOPTYMALIZOWANY kod - cache atrybutów jako zmienne lokalne:
rows = self.rows # lokalna kopia - dostęp bez słownika
cols = self.cols
board = self.board
win_row = rows - 1 if player == 'B' else 0

for r in range(rows): # używamy 'rows', nie 'self.rows'
    for c in range(cols):
        if board[r][c] == player: # 'board', nie 'self.board'
            # ... reszta logiki
```

7.4 Early exit w metodzie is_winner()

Sprawdzenie wygranej przez zliczenie wszystkich pionków wroga (`sum(row.count(...))`) zawsze przechodzi przez całą planszę — 64 komórki. W deep search algorytm wywołuje `is_winner()` miliony razy.

state.py — Optymalizacja 4: Early Exit w is_winner()

```
# state.py - is_winner()

def is_winner(self, player):
    # Sprawdzenie dotarcia do krawędzi (O(n)):
    if player == 'B' and 'B' in self.board[self.rows - 1]: return True
    if player == 'W' and 'W' in self.board[0]: return True

    # Sprawdzenie czy wróg ma jakiegokolwiek pionki - EARLY EXIT:
    enemy = 'W' if player == 'B' else 'B'
    for row in self.board:
        if enemy in row: # znaleziono pierwszego wroga
            return False # ← NATYCHMIASTOWE wyjście z pętli
    return True # brak wrogów = wygraliśmy
```

Wpływ 4 optymalizacji na czas wykonania

Porównanie wyjścia STDERR: czas i węzły przed optymalizacjami vs. po optymalizacjach (różnica w sekundach i liczbie operacji)

```
(base) PS C:\Users\Maciek\Documents\AI_pwr\zad2-breakthrough> python game.py
  W _ _ _ _ _
  o _ _ _ _ _ B
  _ _ _ _ _ B
B _ B B B B _
  _ _ _ _ _ B
  _ _ _ _ _
  _ _ _ _ _
  _ W W W W W
  _ _ _ _ _
-----
Liczba rund: 72
Zwyciezca: W
Odwiedzone wezly: 459939
Czas dzialania: 15.0516s
Agent B - Glebokosc: 4, Strategia: eval_hybrid
-> Wagi: alpha=1.0, beta=0, gamma=0
Agent W - Glebokosc: 4, Strategia: eval_hybrid
-> Wagi: alpha=2.0, beta=50.0, gamma=1.5
(base) PS C:\Users\Maciek\Documents\AI_pwr\zad2-breakthrough> python game.py
  W _ _ _ _ _
  o _ _ _ _ _ B
  _ _ _ _ _ B
B _ B B B B _
  _ _ _ _ _ B
  _ _ _ _ _
  _ _ _ _ _
  _ W W W W W
  _ _ _ _ _
-----
Liczba rund: 72
Zwyciezca: W
Odwiedzone wezly: 459939
Czas dzialania: 10.1500s
Agent B - Glebokosc: 4, Strategia: eval_hybrid
-> Wagi: alpha=1.0, beta=0, gamma=0
Agent W - Glebokosc: 4, Strategia: eval_hybrid
-> Wagi: alpha=2.0, beta=50.0, gamma=1.5
```

8. Wersja rozszerzona — dodatkowe funkcjonalności

8.1 Pojedynki agentów (Dual Agent Mode)

W wersji rozszerzonej gra obsługuje potyczki między dwoma niezależnymi agentami, z których każdy może mieć inną heurystykę i inny parametr głębokości. Konfiguracja odbywa się bezpośrednio w pliku `game.py`:

game.py — Konfiguracja dual-agent mode

```
# game.py - przykładowe konfiguracje agentów:
agent1 = MinimaxAgent('B', max_depth=5, heuristic_func=eval_adaptive)
agent2 = MinimaxAgent('W', max_depth=5, heuristic_func=eval_race)

# Lub z własnym zestawem wag dla eval_hybrid:
agent1 = MinimaxAgent('B', max_depth=3, heuristic_func=eval_hybrid,
                      alpha=0.34, beta=89.84, gamma=0.09, delta=8.42)
```

8.2 Graficzny interfejs użytkownika (gui2.py)

Moduł `gui2.py` implementuje wizualizację w bibliotece `tkinter`. Idea: najpierw prekalkulowana jest cała gra (wszystkie ruchy), a następnie użytkownik może ją odtworzyć krok po kroku, obserwując:

- Aktualny stan planszy z oznaczeniem ostatniego ruchu (kolor pomarańczowy)
- Czas wyliczenia danego ruchu przez AI i liczbę odwiedzonych węzłów
- Bieżącą ocenę heurystyczną (`eval_hybrid`) z perspektywy obu graczy
- Numer ruchu i informację o gracz, który wykonuje bieżący ruch

8.3 Symulator optymalizacji wag (optimize_weights.py)

Moduł umożliwia turniejowe wyszukiwanie optymalnych wag dla `eval_hybrid`. Metodologia:

1. Losowanie N zestawów wag (`alpha`, `beta`, `gamma`, `delta`) z zdefiniowanych przedziałów
2. Rozegranie turnieju każdy z każdym: każda para gra 2 mecze (raz jako B, raz jako W)
3. Zliczanie zwycięstw każdej konfiguracji i sortowanie wyników malejąco
4. Wypisanie rankingu z najlepszą znaną konfiguracją

optimize_weights.py — Generowanie losowych konfiguracji wag

```
# optimize_weights.py - generowanie kandydatów:
candidates.append({
    'alpha': round(random.uniform(0.0, 5.0), 2), # materiał
    'beta': round(random.uniform(10.0, 100.0), 2), # wyścig
    'gamma': round(random.uniform(0.0, 5.0), 2), # presja
    'delta': round(random.uniform(0.0, 10.0), 2), # zagrożenie
})
```

Wynik algorytmu turniejowego dla 10 losowych konfiguracji wag evaluatora hybrydowego

```
[89/90] Kandydat 10 (B) vs Kandydat 8 (W) ... Wygral B (Kandydat 10)  
[90/90] Kandydat 10 (B) vs Kandydat 9 (W) ... Wygral B (Kandydat 10)
```

```
=== WYNIKI SYMULACJI ===
```

```
1. Wygrane: 17/18 | alpha=0.34, beta=89.84, gamma=0.09, delta=8.42 (ID: 6)  
2. Wygrane: 11/18 | alpha=1.34, beta=11.54, gamma=4.82, delta=9.4 (ID: 9)  
3. Wygrane: 8/18 | alpha=0.65, beta=77.93, gamma=3.77, delta=2.51 (ID: 2)  
4. Wygrane: 8/18 | alpha=2.5, beta=36.09, gamma=1.11, delta=0.41 (ID: 4)  
5. Wygrane: 8/18 | alpha=2.88, beta=35.49, gamma=1.46, delta=5.72 (ID: 5)  
6. Wygrane: 8/18 | alpha=4.88, beta=48.2, gamma=3.09, delta=1.2 (ID: 7)  
7. Wygrane: 8/18 | alpha=4.55, beta=63.41, gamma=4.34, delta=4.04 (ID: 8)  
8. Wygrane: 8/18 | alpha=1.47, beta=69.09, gamma=2.42, delta=8.34 (ID: 10)  
9. Wygrane: 7/18 | alpha=1.47, beta=82.25, gamma=1.4, delta=9.72 (ID: 1)  
10. Wygrane: 7/18 | alpha=2.73, beta=79.81, gamma=4.78, delta=2.14 (ID: 3)
```

```
Najlepsza znaleziona konfiguracja: {'id': 6, 'alpha': 0.34, 'beta': 89.84, 'gamma': 0.09, 'delta': 8.42, 'wins': 17}
```


9. Wyniki eksperymentalne i anomalie

9.1 Anomalia: głębokość a wynik meczu (eval_race vs eval_adaptive)

Zaobserwowano zaskakujące zachowanie przy zmianie parametru głębokości d:

Konfiguracja	Wynik	Wyjaśnienie
eval_adaptive (B, d=4) vs eval_race (W, d=4)	Wygrywa W (eval_race)	eval_race przy d=4 konsekwentnie wybiera krótszą ścieżkę do mety szybciej, niż adaptacyjna strategia zdąży zmienić priorytety
eval_adaptive (B, d=5) vs eval_race (W, d=5)	Wygrywa B (eval_adaptive)	Przy d=5 adaptacyjna heurystyka ma czas 'zobaczyć' punkt krytyczny i przełączyć wagi — przebija prostą strategię wyścigową

Wniosek: eval_adaptive wymaga większej głębokości przeszukiwania, by w pełni wykorzystać dynamiczne wagi. Przy d=4 zbyt rzadko widzi punkt, w którym opłaca się zignorować materiał i ruszyć do mety — co kontrybuje prosta, konsekwentna eval_race.



Zrzut ekranu #3 — Anomalia głębokości (eval_race d4 wygrywa z eval_adaptive d4, ale d5 odwraca wynik)

Dwa uruchomienia game.py: wyjście STDOUT (stan końcowy planszy + zwycięzca) dla d=4 i d=5

```
(base) PS C:\Users\Maciek\Documents\AI_pwr\zad2-breakthrough> python game.py
_ _ B B B B B
B W B B B B B
_ _ _ _ _
_ _ _ _ _
_ _ _ _ _
o _ _ W W W W W
B _ W W W W W
_ _ _ _ _
Liczba rund: 33
Zwyciezca: B
Odwiedzone wezly: 5728676
Czas dzialania: 234.8879s
Agent B - Glebokosc: 5, Strategia: eval_adaptive
Agent W - Glebokosc: 5, Strategia: eval_race
(base) PS C:\Users\Maciek\Documents\AI_pwr\zad2-breakthrough> python game.py
W _ _ B B B B B
o _ B B B B B
B B _ _ _ _ _
_ B _ _ _ _ _
_ _ _ _ _
_ W W W W W W W
_ W W W W W W W
_ _ _ _ _
Liczba rund: 22
Zwyciezca: W
Odwiedzone wezly: 152252
Czas dzialania: 3.7301s
Agent B - Glebokosc: 4, Strategia: eval_adaptive
Agent W - Glebokosc: 4, Strategia: eval_race
```

9.2 Anomalia: eval_race B (d=4) vs eval_adaptive W (d=4) — czas i węzły

Przy zamianie ról (eval_race gra jako B, eval_adaptive jako W) — gracz B (eval_race) wygrywa, ale płaci za to znacznie wyższym kosztem obliczeniowym:

Agent	Liczba węzłów	Czas
eval_race B (d=4)	~2× więcej węzłów	~4× więcej czasu
eval_adaptive W (d=4)	mniej węzłów	krótszy czas per ruch
Wynik meczu	Wygrywa B (eval_race)	Pomimo wygranej — duży narzut

Wyjaśnienie: eval_race generuje trudniejsze drzewo do przycięcia — kolejność ruchów jest mniej przewidywalna, co zmniejsza efektywność cięć alfa-beta. eval_adaptive, dzięki wieloskładnikowej ocenie, wytwarza wyraźniejszy gradient, który cięcia alfa-beta lepiej eksploatują.

Zrzut ekranu #4 — eval_race B wygrywa z eval_adaptive W, ale zajmuje 4× więcej czasu

Wyjście STDERR: odwiedzone węzły i czas dla obu agentów — widoczna asymetria kosztów obliczeniowych

```
(base) PS C:\Users\Maciek\Documents\AI_pwr\zad2-breakthrough> python game.py
W _ _ B B B B
o _ B B B B B
B B _ _ _ _
_ B _ _ _ _
_ _ _ _ _ _
_ W W W W W W
_ W W W W W W
-----
Liczba rund: 22
Zwyciezca: W
Odwiedzone wezly: 152252
Czas dzialania: 3.7301s
Agent B - Glebokosc: 4, Strategia: eval_adaptive
Agent W - Glebokosc: 4, Strategia: eval_race
(base) PS C:\Users\Maciek\Documents\AI_pwr\zad2-breakthrough> python game.py
W _ _ _ B B B
o _ _ B B B B
_ B B B _ _ _
_ _ _ _ _ _
B _ _ W _ _ _
W _ _ _ _ _
_ _ W _ _ W W W
_ _ _ W W W W W
-----
Liczba rund: 44
Zwyciezca: W
Odwiedzone wezly: 389027
Czas dzialania: 14.7565s
Agent B - Glebokosc: 4, Strategia: eval_race
Agent W - Glebokosc: 4, Strategia: eval_adaptive
```

10. Napotkane problemy implementacyjne

10.1 Efekt odwlekania wygranej (Horizon Effect)

Problem: algorytm przypisywał wygranej stałą wartość 99999, niezależnie od odległości. Powodowało to, że AI wolało najpierw zbić pionka wroga (po drodze), a dopiero potem wejść na metę — oba warianty miały identyczną ocenę końcową.

- Symptom: AI nie kończy gry natychmiast, gdy ma otwartą drogę do mety
- Rozwiązanie: premia $\text{depth} * 1000$ dodawana do wygranych stanów — szybsza wygrana = wyższy wynik

10.2 Narzut obliczeniowy w ewaluacji hybrydowej

Problem: nawet przy wagach zerowych dla niektórych składowych `eval_hybrid`, system iterował po całej planszy wywołując wszystkie heurystyki.

- Symptom: `eval_hybrid` z wagami (0, 1, 0, 0) działał tak samo wolno jak z (1, 1, 1, 1)
- Rozwiązanie: dodano warunki `if alpha != 0, if beta != 0` itp. — pomijanie całych iteracji dla zerowych wag

10.3 Zarządzanie znacznikiem 'o' w drzewie stanu

Problem: początkowo przed każdym wykonaniem ruchu algorytm przeszukiwał całą planszę 8×8 w poszukiwaniu znaku 'o' do usunięcia. Przy głębokości $d=5$ z branching factor ~ 18 , generuje to miliony zbędnych przeszukań.

- Symptom: profilowanie kodu wskazywało `_create_new_state` jako hot-spot
- Rozwiązanie: atrybut `last_origin = (r, c)` przechowuje pozycję poprzedniego 'o' — usunięcie w $O(1)$

11. Literatura i biblioteki

11.1 Materiały źródłowe

1. Wikipedia: Breakthrough (board game) — zasady gry
2. Materiały wykładowe: Sztuczna Inteligencja i Inżynieria Wiedzy, PWr 2026

11.2 Wykorzystane biblioteki Pythona

Biblioteka	Typ	Zastosowanie
math	Standardowa	Stałe <code>math.inf</code> i <code>-math.inf</code> jako wartości startowe α i β w algorytmie Minimax
time	Standardowa	Pomiar czasu wykonania każdego ruchu (wyjście <code>STDERR</code>)
sys	Standardowa	Wczytywanie planszy ze <code>stdin</code> , wyjście błędów na <code>stderr</code>
inspect	Standardowa	Dynamiczne odczytywanie sygnatur funkcji (wypisywanie wag agentów)
random	Standardowa	Generowanie losowych zestawów wag w symulatorze turniejowym
tkinter	Standardowa	Graficzny interfejs użytkownika — wizualizacja planszy i statystyk